

NexuMathEdu

Documentación Técnica del Sistema

Plataforma de Gestión Educativa con IA

Autor: Equipo NexuMathEdu

Versión: 1.0.0 (Estable)

Fecha de publicación: Junio, 2026

Contexto de Rutas para IA — NexuMathEdu

Este documento describe todas las rutas y endpoints de la aplicación NexuMathEdu, útil para dar contexto a una IA sobre la estructura del sistema.

1. Tech Stack

Capa	Tecnología
Frontend	React 19 + Vite 8
Lenguaje	JavaScript (JSX), TypeScript en Edge Functions
Routing	React Router DOM v7
Estilos	Tailwind CSS v4
Backend/Datos	Supabase (PostgreSQL + Auth + Edge Functions)
IA Chat	GROQ + Cerebras APIs (modelo `gpt-oss-120b` en Profesor 2)

2. Roles del sistema

Rol	Descripción
`admin`	Administrador del sistema — gestión completa de usuarios, cursos, asignaciones
`teacher`	Profesor — gestión de sus propios cursos, estudiantes y calificaciones
`student`	Estudiante — consulta de cursos y calificaciones, chat con tutor IA

3. Rutas del frontend (React Router)

Ruta	Componente	Acceso
`/login`	`Login.jsx`	Público
`/`	Redirección según rol	Autenticado
`/admin`	`AdminDashboard.jsx`	`admin`
`/teacher`	`TeacherDashboard.jsx`	`teacher`
`/student`	`StudentDashboard.jsx`	`student`
`/perfil`	`ProfilePage.jsx`	Cualquier rol autenticado
`/chat`	`ChatPage.jsx`	Cualquier rol autenticado
`**`	Redirección al dashboard del rol	—

4. Edge Functions de Supabase (Backend)

4.1. Usuarios

Función	Método	Ruta	Rol
Listar usuarios	GET	<code>/functions/v1/list-users`</code>	<code>`admin`</code>
Crear usuario	POST	<code>/functions/v1/create-user`</code>	<code>`admin`, `teacher`</code>
Actualizar usuario	POST	<code>/functions/v1/update-user`</code>	<code>`admin`</code>
Eliminar usuario	POST	<code>/functions/v1/delete-user`</code>	<code>`admin`</code>

4.2. Cursos y asignaciones

Función	Método	Ruta	Rol
Eliminar curso	POST	<code>/functions/v1/delete-course`</code>	<code>`admin`</code>
Asignar estudiantes a profesor	POST	<code>/functions/v1/admin-assign-students-to-teacher`</code>	<code>`admin`</code>
Asignar cursos a profesor	POST	<code>/functions/v1/admin-assign-courses-to-teacher`</code>	<code>`admin`</code>

4.3. Datos de dashboard

Función	Método	Ruta	Rol
Datos dashboard profesor	POST	<code>/functions/v1/teacher-dashboard-data`</code>	<code>`teacher`</code>
Datos dashboard estudiante	POST	<code>/functions/v1/student-dashboard-data`</code>	<code>`student`</code>

4.4. Chat

Función	Método	Ruta	Rol
Chat con IA	GET/POST	<code>/functions/v1/chat`</code>	Todos

5. Tablas de base de datos (Supabase PostgreSQL)

Tabla	Admin	Teacher	Student
<code>`profiles`</code>	CRUD completo	Crear estudiantes, leer propios	Leer propio
<code>`courses`</code>	CRUD completo	CRUD sobre propios	Leer
<code>`enrollments`</code>	CRUD completo	CRUD sobre matrículas de sus cursos	Leer propias
<code>`course_grades`</code>	CRUD completo	CRUD sobre calificaciones de sus cursos	Leer propias
<code>`assessments`</code>	Escribir	Escribir	Leer

`grade_records`	Escribir	Escribir	Leer propias
`chat_threads`	Leer todas	—	Leer propias
`chat_messages`	Leer todas	—	Leer propias

6. Endpoints de autenticación (Supabase Auth)

Operación	Uso
`signInWithPassword`	Login
`signOut`	Logout (todos los dashboards, perfil)
`getSession`	Inicialización de AuthContext
`updateUser` (password)	Cambio de contraseña en perfil
`onAuthStateChange`	Listener de estado de autenticación

7. Estructura de directorios del proyecto

```
NexuMathEdu/
+-- src/
|   +-- App.jsx                # Definición de rutas
|   +-- main.jsx               # Punto de entrada
|   +-- components/
|   |   +-- Login.jsx
|   |   +-- ProtectedRoute.jsx # Guard de rutas por rol
|   |   +-- dashboard/
|   |   |   +-- CourseModal.jsx
|   |   |   +-- CoursePerformanceSection.jsx
|   |   |   +-- EnrollmentModal.jsx
|   |   |   +-- TeacherAssignmentModal.jsx
|   |   |   +-- UserModal.jsx
|   |   +-- layout/
|   |   |   +-- DashboardHeader.jsx
|   |   |   +-- PageLoader.jsx
|   |   |   +-- ProtectedLayout.jsx
|   |   +-- chat/
|   |   |   +-- ChatComposer.jsx
|   |   |   +-- ChatMessageBubble.jsx
|   |   |   +-- ChatSettingsPanel.jsx
|   |   |   +-- ChatThreadList.jsx
|   |   |   +-- MarkdownContent.jsx
|   +-- constants/
|   |   +-- gradeLevels.js
|   +-- contexts/
|   |   +-- AuthContext.jsx
|   +-- lib/
|   |   +-- supabase.js
|   |   +-- chatClient.js
|   +-- pages/
```

```
| | +-- AdminDashboard.jsx
| | +-- TeacherDashboard.jsx
| | +-- StudentDashboard.jsx
| | +-- ProfilePage.jsx
| | +-- ChatPage.jsx
| +-- utils/
|   +-- roleRoutes.js
|   +-- chatPresets.js
|   +-- courseAnalytics.js
|   +-- grades.js
|   +-- withTimeout.js
+-- supabase/
| +-- schema.sql
| +-- config.toml
| +-- functions/
|   +-- list-users/
|   +-- create-user/
|   +-- update-user/
|   +-- delete-user/
|   +-- delete-course/
|   +-- admin-assign-students-to-teacher/
|   +-- admin-assign-courses-to-teacher/
|   +-- teacher-dashboard-data/
|   +-- student-dashboard-data/
|   +-- chat/
+-- docs/
  +-- manual-admin.md
  +-- manual-profesor.md
  +-- manual-estudiante.md
  +-- rutas-contexto-ia.md
```

8. Resumen de capacidades por rol

Admin

- CRUD de usuarios (admin, teacher, student)
- CRUD de cursos
- Asignaciones masivas (estudiantes → profesor, cursos → profesor)
- Matriculación de estudiantes en cursos
- Visualización de rendimiento global
- Chat con IA (perfil admin)

Teacher

- CRUD de sus propios cursos
- Creación de estudiantes
- Matriculación de estudiantes en sus cursos
- Ingreso de calificaciones (P1, P2, P3) — PF automática
- Visualización de rendimiento de sus cursos
- Chat con IA (perfil profesor)

Student

- Consulta de cursos y calificaciones (solo lectura)
- Visualización de estadísticas personales
- Chat con IA (perfil tutor)

9. Esquema Detallado de la Base de Datos (PostgreSQL)

Este esquema describe las tablas, tipos de datos, llaves foráneas, triggers, índices y políticas de seguridad (RLS) en la base de datos de Supabase.

9.1. Tipos y Tipos de Datos Personalizados

- `user_role`: Tipo enumerado `ENUM ('admin', 'teacher', 'student')`.

9.2. Tablas del Sistema

9.2.1. Tabla `profiles`

Almacena la información de perfil para todos los usuarios.

- `id` (`uuid`, PK): Referencia a `auth.users(id)` con eliminación en cascada.
- `email` (`text`, unique, no nulo): Correo electrónico del usuario.
- `full_name` (`text`): Nombre completo.
- `role` (`user_role`, no nulo, default `'student'`): Rol de acceso.
- `avatar_url` (`text`): URL de la imagen de perfil.
- `grade_level` (`text`): Grado académico asignado (estudiante).
- `assigned_grade_levels` (`text[]`, default `'{}'`): Grados académicos asignados (profesor).
- `chat_provider` (`text`, default `'profesor_1'`): Proveedor predeterminado de IA.
- `created_by` (`uuid`): Referencia a `profiles(id)` con eliminación en nulo.
- `created_at` / `updated_at` (`timestampz`).

Triggers asociados:

- `on_auth_user_created`: Ejecuta `public.handle_new_user()` tras la inserción en `auth.users`.
- `set_profiles_updated_at`: Ejecuta `public.set_updated_at()` antes de actualizar.
- `prevent_invalid_profile_role_changes_on_profiles`: Ejecuta `public.prevent_invalid_profile_role_changes()` antes de actualizar.

Políticas RLS:

- `profiles_select_own` (SELECT): Permitido si el usuario lee su propio perfil, es administrador, o es profesor y el perfil fue creado por él.
- `profiles_update_own` (UPDATE): Permitido solo para actualizar su propio perfil.
- `profiles_admin_insert` (INSERT): Permitido si el usuario crea su propio perfil o es administrador.
- `profiles_admin_delete` (DELETE): Permitido solo para administradores (no pueden eliminarse a sí mismos).

9.2.2. Tabla `courses`

Almacena los cursos creados en la plataforma.

- `id` (`uuid`, PK): Generado automáticamente (`gen_random_uuid()`).
- `title` (`text`, no nulo): Título del curso.

- `description (text)`: Descripción detallada.
- `subject (text)`: Materia del curso.
- `grade_level (text)`: Grado al que pertenece.
- `teacher_id (uuid)`: Referencia a `profiles(id)` con eliminación en nulo.
- `is_active (boolean, default true)`.
- `created_at / updated_at (timestampz)`.

Triggers asociados:

- `set_courses_updated_at`: Ejecuta `public.set_updated_at()` antes de actualizar.

Políticas RLS:

- `courses_select_authenticated` (SELECT): Permitido para todos los usuarios autenticados.
- `courses_write_admin_teacher` (INSERT): Permitido para administradores o profesores dueños del curso.
- `courses_update_admin_teacher` (UPDATE): Permitido para administradores o profesores dueños del curso.
- `courses_delete_admin_teacher` (DELETE): Permitido para administradores o profesores dueños del curso.

9.2.3. Tabla ``enrollments``

Vincula estudiantes matriculados en cursos.

- `id (uuid, PK)`.
- `course_id (uuid, no nulo)`: Referencia a `courses(id)` con eliminación en cascada.
- `student_id (uuid, no nulo)`: Referencia a `profiles(id)` con eliminación en cascada.
- `enrolled_at (timestampz)`.
- Restricción única en `(course_id, student_id)`.

Políticas RLS:

• `enrollments_select` (SELECT): Estudiantes ven sus propias matrículas, profesores ven matrículas en sus cursos, y administradores ven todas.

- `enrollments_write_admin_teacher` (INSERT): Administradores o profesores de ese curso.
- `enrollments_update_admin_teacher` (UPDATE): Administradores o profesores de ese curso.
- `enrollments_delete_admin_teacher` (DELETE): Administradores o profesores de ese curso.

9.2.4. Tabla ``assessments``

Tareas, actividades o exámenes de los cursos.

- `id (uuid, PK)`.
- `course_id (uuid)`: Referencia a `courses(id)` con eliminación en cascada.
- `title (text, no nulo)`.
- `description (text)`.
- `max_score (numeric(5,2), default 5.00)`.
- `due_date (date)`.
- `created_by (uuid)`: Referencia a `profiles(id)`.
- `created_at / updated_at (timestampz)`.

Triggers asociados:

- `set_assessments_updated_at`: Actualiza `updated_at`.

Políticas RLS:

- `assessments_select` (SELECT): Permitido a todos los usuarios autenticados.

- `assessments_write_admin_teacher` (INSERT): Profesores y administradores.

9.2.5. Tabla ``grade_records``

Notas detalladas de evaluaciones individuales.

- `id` (`uuid`, PK).
- `assessment_id` (`uuid`): Referencia a `assessments(id)` con eliminación en cascada.
- `student_id` (`uuid`): Referencia a `profiles(id)` con eliminación en cascada.
- `score` (`numeric(5,2)`, no nulo, ≥ 0).
- `feedback` (`text`).
- `graded_by` (`uuid`): Referencia a `profiles(id)`.
- `graded_at` (`timestampz`).
- Restricción única en (`assessment_id`, `student_id`).

Políticas RLS:

- `grade_records_select` (SELECT): Estudiante dueño de la nota, profesores del curso correspondiente, y administradores.
- `grade_records_write_admin_teacher` (INSERT): Profesores y administradores.
- `grade_records_update_admin_teacher` (UPDATE): Profesores y administradores.

9.2.6. Tabla ``course_grades``

Calificaciones periódicas e integradas para el boletín.

- `id` (`uuid`, PK).
- `course_id` (`uuid`): Referencia a `courses(id)` en cascada.
- `student_id` (`uuid`): Referencia a `profiles(id)` en cascada.
- `note_1` (`numeric(5,2)`, default 0).
- `note_2` (`numeric(5,2)`, default 0).
- `note_3` (`numeric(5,2)`, default 0).
- `final_grade` (`numeric(5,2)`, default 0): Nota final.
- `updated_by` (`uuid`): Referencia a `profiles(id)`.
- `created_at` / `updated_at` (`timestampz`).
- Restricción única en (`course_id`, `student_id`).

Triggers asociados:

- `set_course_grades_updated_at`: Actualiza `updated_at`.
- `set_course_grades_final_grade`: Ejecuta `public.set_course_grades_final_grade()`. Calcula y redondea de manera automática la nota final: $PF = (note_1 + note_2 + note_3) / 3$.

Políticas RLS:

- `course_grades_select` (SELECT): Estudiante dueño, profesores de sus cursos, y administradores.
- `course_grades_write_admin_teacher` (INSERT): Profesores del curso y administradores.
- `course_grades_update_admin_teacher` (UPDATE): Profesores del curso y administradores.

9.2.7. Tabla ``chat_threads``

Hilos de conversación de chat con IA.

- `id` (`uuid`, PK).
- `student_id` (`uuid`): Referencia a `profiles(id)` en cascada.

- `title` (`text`, default `'Chat'`).
- `provider` (`text`, default `'profesor_1'`).
- `tone` (`text`, default `'claro'`).
- `detail_level` (`text`, default `'medio'`).
- `focus` (`text`, default `'matematicas'`).
- `language` (`text`, default `'espanol'`).
- `topic` (`text`, default `'general'`).
- `created_at` / `updated_at` (`timestampz`).

Triggers asociados:

- `set_chat_threads_updated_at`: Actualiza `updated_at`.

Políticas RLS:

- `chat_threads_select_owner` (SELECT): Dueño del hilo, administradores, y profesores de cursos en los que el estudiante esté matriculado.
- `chat_threads_write_owner` (INSERT): Permitido si el `student_id` es el ID del usuario autenticado.

9.2.8. Tabla `chat_messages``

Mensajes detallados pertenecientes a un hilo de chat.

- `id` (`uuid`, PK).
- `thread_id` (`uuid`): Referencia a `chat_threads(id)` en cascada.
- `sender_role` (`text`, no nulo): Rol del emisor (`student`, `assistant`, `teacher`, `admin`).
- `content` (`text`, no nulo): Contenido del mensaje.
- `created_at` (`timestampz`).

Políticas RLS:

- `chat_messages_select_owner` (SELECT): Permitido a dueños del hilo, administradores y profesores del estudiante correspondiente.
- `chat_messages_write_owner` (INSERT): Permitido al estudiante dueño del hilo.

9.3. Funciones y Procedimientos Almacenados (PL/pgSQL)

- `set_updated_at()`: Trigger helper para actualizar de forma automática la columna `updated_at` con `now()`.
- `set_course_grades_final_grade()`: Trigger helper que calcula la nota final `final_grade` promediando y redondeando las tres notas (`note_1`, `note_2`, `note_3`).
- `handle_new_user()`: Se ejecuta en el trigger `on_auth_user_created` para copiar la información del nuevo usuario de `auth.users` a `public.profiles` con su rol adecuado (`admin`, `teacher` o `student`).
- `current_user_role()`: Extrae el rol actual del usuario desde los metadatos de su token JWT (`app_metadata` o `user_metadata`), o realiza una consulta directa a `auth.users` si el JWT no lo posee.
- `has_any_role(allowed_roles user_role[])`: Verifica si el rol actual del usuario autenticado está incluido dentro de un listado de roles autorizados.
- `prevent_invalid_profile_role_changes()`: Restringe cambios de roles en perfiles, previniendo que los profesores alteren roles y asegurando que solo los administradores manejen roles elevados.

10. Prompts de Sistema para el Chat con IA

El comportamiento del tutor inteligente de NexuMathEdu está controlado por prompts de sistema generados dinámicamente según el rol del usuario autenticado y su configuración de chat actual.

10.1. Instrucciones Específicas por Rol del Usuario

- **Rol Administrador (admin):**

Eres un asistente de soporte operativo y técnico para administradores de NexuMathEdu. Tus respuestas deben ser precisas, breves y orientadas a la gestión de la plataforma y resolución de incidentes técnicos.

- **Rol Profesor (teacher):**

Eres un asesor pedagógico para profesores en NexuMathEdu. Ayúdales a diseñar planes de clase, rúbricas de evaluación, estructurar problemas matemáticos dinámicos y sugerir actividades didácticas. Mantén un enfoque profesional, didáctico y propositivo.

- **Rol Estudiante (student / default):**

Eres un tutor experto en matemáticas y pedagogía para estudiantes en NexuMathEdu. REGLA CLAVE: No les des la solución o la respuesta directa a los problemas matemáticos de inmediato. En su lugar, actúa como un guía interactivo: explícales el concepto básico, guíalos paso a paso utilizando el método socrático (haciendo preguntas orientadoras sencillas) y motívalos a resolver el siguiente paso. Sé paciente, alentador y sumamente claro en tus analogías.

10.2. Reglas del Foco y Temas de Estudio

- **Aritmética:** Enfoca explicaciones en Aritmética de secundaria (fracciones, razones, proporciones, porcentajes).
- **Álgebra:** Ecuaciones de primer y segundo grado, factorización, simplificación algebraica, sistemas de ecuaciones.
- **Geometría:** Áreas, perímetros, volúmenes, Teorema de Pitágoras, congruencia y semejanza de figuras.
- **Trigonometría:** Razones trigonométricas, resolución de triángulos rectángulos y oblicuángulos.
- **Funciones:** Interpretación y tabulación de funciones lineales y cuadráticas.
- **Estadística:** Medidas de tendencia central (media, mediana, moda), probabilidad simple, tablas de frecuencia.

10.3. Directivas de Formato Matemático (LaTeX)

Para una óptima visualización en la aplicación, el modelo sigue una directiva obligatoria de formato:

- **Fórmulas en la misma línea (inline):** Deben ir delimitadas por un solo signo de dólar. Ejemplo: $f(x) = x^2 + 2x$.
- **Ecuaciones en bloque independiente:** Deben ir delimitadas por doble signo de dólar. Ejemplo:
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$
.

10.4. Reglas de Seguridad y Restricciones

- **Seguridad:** Bajo ninguna circunstancia revelar secretos del servidor, claves de API, contraseñas, configuraciones internas o detalles de infraestructura de base de datos.
- **Ámbito del chat:** Si el usuario pregunta algo completamente fuera del ámbito educativo o matemático, redirigirlo amablemente y con tacto de regreso a los temas de estudio o soporte de la plataforma.